SENAI

SERVIÇO NACIONAL DE APRENDIZAGEM INDUSTRIAL

ATIVIDADE 02 — RELATÓRIO TÉCNICO

REACT

"Características, vantagens, aplicações no mercado e exemplo de utilização em um projeto Web"

Curso: Análise e Desenvolvimento de Sistemas

Aluno: Nicolas Pureza

Turma: _____

Data: ____/____/2026

Sorocaba – SP

2026

1. INTRODUÇÃO E HISTÓRICO

O desenvolvimento Front-end compreende a construção da interface de usuário (UI) de aplicações digitais, sendo a camada diretamente acessível e interativa para os usuários finais. No ecossistema Web tradicional, a tríade fundamental de tecnologias é composta pelo HTML (*HyperText Markup Language*), responsável pela estruturação e semântica do conteúdo; pelo CSS (*Cascading Style Sheets*), incumbido da estilização visual e do layout; e pelo JavaScript, linguagem de programação responsável por conferir dinamismo, interatividade e manipulação do Modelo de Objeto do Documento (DOM — *Document Object Model*).

Com o avanço das demandas de mercado por aplicações cada vez mais complexas, responsivas e em tempo real — conhecidas como *Single Page Applications* (SPAs) —, a manipulação direta e manual do DOM via JavaScript puro (Vanilla JS) tornou-se uma tarefa propensa a falhas, de difícil manutenção e computacionalmente ineficiente. A necessidade de sincronizar constantemente o estado da aplicação com a interface visual impulsionou o surgimento de bibliotecas e *frameworks* Front-end modernos, os quais introduziram novos paradigmas de arquitetura de software.

Neste cenário, o **React** consolida-se como uma das ferramentas mais influentes e adotadas no ecossistema de desenvolvimento de software moderno. Desenvolvido originalmente pelo Facebook (atual Meta), o React é uma biblioteca JavaScript de código aberto focada na construção de interfaces de usuário reativas e baseadas em componentes declarativos.

A gênese do React remonta ao ano de 2011, quando o engenheiro de software Jordan Walke criou o protótipo *FaxJS* para resolver problemas de desempenho e complexidade no gerenciamento de atualizações da interface do Feed de Notícias do Facebook. Em 2012, após a aquisição do Instagram, a ferramenta foi aprimorada e aplicada à plataforma de fotos. Em maio de 2013, durante a conferência JSConf US, o React teve seu código aberto à comunidade global de desenvolvedores. Desde então, sob o respaldo contínuo da Meta e de uma vibrante comunidade *open-source*, a ferramenta evoluiu significativamente, passando pela transição dos componentes baseados em classe para componentes funcionais equipados com *Hooks* (versão 16.8) e culminando na consolidação da **React Foundation** em 2026, entidade voltada à governança neutra e sustentável da ferramenta.

1.1 Objetivo do Relatório

Este relatório técnico tem por objetivo apresentar uma análise aprofundada da biblioteca React, investigando suas características fundamentais, suas vantagens operacionais e limitações, suas principais aplicações no mercado corporativo e a demonstração prática de sua arquitetura por meio do planejamento e codificação de um projeto Web de Catálogo de Filmes.

1.2 Importância do Tema

Para estudantes do curso de Análise e Desenvolvimento de Sistemas (ADS), o domínio do React é de suma relevância estratégica. Compreender a mecânica interna do React prepara o futuro

analista de software para responder às exigências reais da indústria de tecnologia, na qual a alta demanda por sistemas escaláveis e reutilizáveis estabeleceu o React como padrão de mercado.

2. PRINCIPAIS CARACTERÍSTICAS DO REACT

A popularização acelerada do React decorre de escolhas arquiteturais inovadoras que revolucionaram a forma como interfaces de usuário são projetadas e construídas. A seguir, detalham-se seus pilares estruturais.

2.1 Componentização

A componentização é o conceito central do React. Ela consiste em fragmentar a interface de usuário em peças isoladas, independentes e reutilizáveis chamadas **componentes**. Cada componente engloba sua própria estrutura HTML, lógica de funcionamento em JavaScript e estilização. Essa abordagem modular facilita substancialmente a manutenção, os testes unitários e a legibilidade do código-fonte.

2.2 JSX (JavaScript XML)

O JSX é uma extensão de sintaxe para JavaScript que permite escrever estruturas semelhantes ao HTML diretamente dentro do código JavaScript. O JSX não é interpretado nativamente pelos navegadores; portanto, durante a etapa de compilação (através de ferramentas como Babel ou Vite), o JSX é transformado em chamadas puras da API do React (`React.createElement`). O uso do JSX oferece segurança contra ataques de *Cross-Site Scripting* (XSS) e expressividade no desenvolvimento.

2.3 Props (Propriedades)

As *props* são argumentos passados para os componentes React, funcionando de maneira análoga aos parâmetros de uma função em linguagens de programação. No React, o fluxo de dados é estritamente unidirecional (de pai para filho). As *props* são imutáveis do ponto de vista do componente receptor, garantindo previsibilidade e consistência na renderização.

2.4 State (Estado) e Hooks

Enquanto as *props* transmitem dados externos, o **Estado** (*State*) representa a memória interna de um componente. Quando o estado de um componente é alterado, o React automaticamente re-renderiza o componente na tela. Os **Hooks**, introduzidos no React 16.8, são funções especiais que permitem ao desenvolvedor utilizar o estado e outros recursos do ciclo de vida do React dentro de componentes funcionais. O Hook fundamental para controle de estado é o `useState`, complementado por Hooks como `useEffect` (efeitos colaterais) e `useContext` (compartilhamento global de dados).

2.5 Renderização Declarativa e Virtual DOM

O React adota um paradigma de programação declarativo. Em vez de descrever passo a passo como alterar o DOM para refletir uma mudança (paradigma imperativo), o desenvolvedor apenas declara qual deve ser a aparência da UI para um dado estado. O React gerencia as

alterações reais através do **Virtual DOM** — uma representação leve e em memória do DOM real. Quando ocorrem mudanças de estado, o React compara o Virtual DOM atual com o anterior através de um algoritmo altamente eficiente de *diffing* e aplica no DOM real apenas as alterações estritamente necessárias (*reconciliation*).

2.6 Exemplo Prático de Código

Abaixo, demonstra-se a criação de um componente reutilizável e sua composição na aplicação:

```
// Componente filho reutilizável
function Botao() {
  return <button className="btn-primary">Comprar</button>;
}

// Componente pai principal
export default function App() {
  return (
    <div className="container">
      <h1>Loja Online SENAI</h1>
      <p>Confira nossos produtos disponíveis:</p>
      <Botao />
    </div>
  );
}
```

3. VANTAGENS E LIMITAÇÕES

Como qualquer decisão de arquitetura no desenvolvimento de sistemas, a adoção do React envolve a avaliação de seus pontos fortes e de suas eventuais restrições técnicas no ciclo de vida de um projeto de software.

3.1 Vantagens

- **Reutilização de Código e Modularidade:** A arquitetura baseada em componentes permite reaproveitar elementos de interface em diferentes páginas ou projetos, reduzindo o tempo de desenvolvimento e a duplicidade de código.
- **Alta Performance com Virtual DOM:** A atualização otimizada da tela minimiza o impacto no processamento do navegador, garantindo navegação fluida em aplicações robustas.
- **Ecossistema Rico e Comunidade Ativa:** Há uma vasta quantidade de bibliotecas de terceiros, roteadores, componentes de UI e ferramentas de testes mantidas pela comunidade global.
- **Adoção Gradual:** O React pode ser inserido pontualmente em projetos legados existentes sem a necessidade de reescrever todo o sistema de uma só vez.
- **Multiplataforma com React Native:** O conhecimento adquirido no React Web é diretamente aproveitável para a criação de aplicativos móveis nativos (iOS e Android) utilizando o React Native.

3.2 Limitações

- **Classificação Tecnológica (Biblioteca vs. Framework):** O React é estritamente uma biblioteca focada na camada de visão (View). Consequentemente, não inclui nativamente soluções integradas para roteamento, gerenciamento global de estado ou requisições HTTP, exigindo a seleção e configuração de pacotes adicionais.
- **Curva de Aprendizagem e Ritmo de Atualizações:** O ecossistema evolui rapidamente, exigindo constante atualização dos desenvolvedores quanto a novas boas práticas, padrões de código e descontinuação de APIs antigas.
- **Forte Dependência do JavaScript Moderno:** O domínio de recursos avançados do ECMAScript 6+ (como desestruturação, *arrow functions*, *promises* e *modules*) é pré-requisito indispensável para a escrita adequada em React.
- **Complexidade no Gerenciamento de Estado Global:** Em projetos de grande porte, o repasse manual de *props* por múltiplos níveis (*prop drilling*) exige a implementação de ferramentas sofisticadas como Redux, Zustand ou Context API, elevando a complexidade do código.

3.3 Regras dos Hooks

Para garantir que o estado interno seja mantido corretamente entre re-renderizações, o React estabelece duas regras estritas para o uso de Hooks:

1. **Chame Hooks apenas no nível superior:** Não chame Hooks dentro de laços de repetição (*loops*), condições (*if/else*) ou funções aninhadas.

2. **Chame Hooks apenas em funções React:** Chame Hooks apenas em componentes funcionais React ou em Hooks customizados.

3.4 React e o Ecossistema de Frameworks

Para suprir a ausência de recursos nativos típicos de *frameworks full-stack*, a comunidade desenvolveu *frameworks* de alto nível construídos sobre o React, tais como **Next.js** e **Remix**. Essas ferramentas adicionam renderização no lado do servidor (SSR), geração de sites estáticos (SSG) e roteamento baseado no sistema de arquivos, consolidando o React como núcleo de soluções corporativas completas.

4. APLICAÇÕES DO REACT NO MERCADO

A flexibilidade e o alto desempenho do React o tornaram a tecnologia de preferência em diversos segmentos da indústria de software global e nacional. A seguir, destacam-se os principais domínios de aplicação.

4.1 Plataformas de E-commerce

Aplicações de comércio eletrônico exigem interfaces altamente dinâmicas, atualização instantânea de carrinhos de compras, filtros em tempo real e processos de *checkout* seguros sem o recarregamento total da página. O React permite construir componentes reativos para exibições de catálogos, controle de estoque visível e galerias interativas de produtos.

4.2 Dashboards e Painéis Analíticos

Painéis administrativos corporativos dependem do processamento contínuo de dados para exibição de gráficos interativos, tabelas dinâmicas, indicadores de desempenho (KPIs) e notificações em tempo real. A capacidade do React de renderizar apenas os elementos gráficos cujos dados mudaram torna-o ideal para plataformas de *Business Intelligence* (BI).

4.3 Sistemas Corporativos e Gestão (ERP/CRM)

Sistemas internos para cadastros complexos, formulários com validação dinâmica, consultas paginadas e gerenciamento de permissões de acesso beneficiam-se diretamente da arquitetura modular do React, padronizando a experiência do usuário e acelerando o desenvolvimento de novas telas.

4.4 Plataformas de Conteúdo e Mídia

Grandes portais de entretenimento e redes sociais utilizam o React para alimentar feeds infinitos de publicações, reprodutores de vídeo integrados, seções de comentários atualizadas ao vivo e mecânicas de engajamento contínuo.

4.5 Aplicações Móveis Multiplataforma

Através do **React Native**, empresas conseguem compartilhar a lógica de negócios desenvolvida em JavaScript/TypeScript entre aplicações Web e aplicativos móveis para iOS e Android, resultando em expressiva redução de custos operacionais e tempo de lançamento no mercado (*time-to-market*).

4.6 Casos Reais de Uso em Grande Escala

A eficiência da ferramenta é comprovada pela sua utilização por corporações globais líderes de tecnologia:

- **Meta (Facebook/Instagram):** Toda a interface Web moderna e os aplicativos móveis são suportados pela arquitetura React.
- **Netflix:** Utiliza o React em suas interfaces de TV e Web para garantir inicialização rápida e navegação fluida em catálogos extensos.
- **Airbnb:** Adotou o React para padronizar seus componentes visuais globais e acelerar a busca e reserva de acomodações.
- **Spotify:** Aplica a biblioteca em seu *player* Web para reprodução e gestão de listas de reprodução dinâmicas.

4.7 Importância para o Mercado de Trabalho

Atualmente, o React é uma das habilidades mais solicitadas em vagas de emprego para desenvolvedores Front-end e Full-stack em todo o mundo. Para o profissional formado em Análise e Desenvolvimento de Sistemas, a proficiência em React abre portas em *startups*, software houses e multinacionais de tecnologia.

5. EXEMPLO DE UTILIZAÇÃO EM UM PROJETO WEB

Para ilustrar a aplicação prática dos conceitos teóricos abordados, apresenta-se a concepção técnica de um projeto Web intitulado **"Catálogo de Filmes"** (*MovieCatalog*). A aplicação destina-se à consulta de títulos cinematográficos, pequisa dinâmica e salvamento de itens favoritos.

5.1 Arquitetura da Estrutura de Arquivos

A organização do projeto segue as melhores práticas da comunidade para aplicações React estruturadas com a ferramenta de compilação Vite:

```
src/
├── components/
│ ├── Header.jsx # Cabeçalho com título e contador de favoritos
│ ├── SearchBar.jsx # Campo de busca para filtragem de títulos
│ ├── MovieCard.jsx # Cartão individual para exibição do filme
│ └── MovieList.jsx # Container para renderização da lista de filmes
├── App.jsx # Componente raiz e gerenciamento de estado global
└── main.jsx # Ponto de entrada da aplicação e montagem no DOM
```

5.2 Componente Funcional com Recebimento de Props

O componente `MovieCard.jsx` é responsável por apresentar as informações de um filme específico, recebendo seus dados via *props*:

```jsx
// src/components/MovieCard.jsx
export default function MovieCard({ title, year, category }) {
  return (
    <article className="movie-card">
      <h2>{title}</h2>
      <p>Ano de Lançamento: {year}</p>
      <p>Gênero: {category}</p>
    </article>
  );
}
```

5.3 Gerenciamento de Estado com o Hook useState

Abaixo, ilustra-se o controle de interatividade do botão de favoritos através do Hook `useState` no componente `FavoriteButton.jsx`:

```jsx
// src/components/FavoriteButton.jsx
import { useState } from 'react';

export default function FavoriteButton() {
  // Declaração do estado 'favorite' inicializado como false
  const [favorite, setFavorite] = useState(false);

  const handleToggleFavorite = () => {
    setFavorite(!favorite);
  };

  return (
    <button
      className={favorite ? "btn-fav-active" : "btn-fav"}
      onClick={handleToggleFavorite}
    >
      {favorite ? "Remover dos Favoritos" : "Adicionar aos Favoritos"}
    </button>
  );
}
```

5.4 Integração com API Externa (Consumo de Dados)

Em um cenário de produção real, o catálogo de filmes não utilizaria dados estáticos. A aplicação consumiria uma API RESTful externa (como a *The Movie Database - TMDB*) utilizando o Hook `useEffect` em conjunto com a função nativa `fetch` ou a biblioteca `axios` para carregar os dados assincronamente durante a inicialização do componente.

6. CONCLUSÃO

O presente relatório técnico permitiu realizar uma análise detalhada e abrangente sobre a biblioteca Front-end React, compreendendo desde suas origens históricas na Meta até sua posição consolidada como padrão de desenvolvimento Web no mercado contemporâneo.

Compreendeu-se que o sucesso do React fundamenta-se em seus pilares arquiteturais: a componentização modular, que eleva a reusabilidade e manutenibilidade do código; o JSX, que proporciona uma sintaxe expressiva e segura; o controle de estado e a revolução trazida pelos *Hooks*; e a alta performance decorrente do gerenciamento declarativo via Virtual DOM.

Embora apresente limitações inerentes à sua natureza de biblioteca — demandando a escolha consciente de ferramentas complementares e um investimento constante no aprendizado do ecossistema —, o React supera tais restrições através de uma comunidade extremamente produtiva e de *frameworks* robustos como o Next.js.

Para os alunos do curso de Análise e Desenvolvimento de Sistemas do SENAI, o estudo teórico e prático do React representa um diferencial competitivo crucial. O desenvolvimento do exemplo prático do "Catálogo de Filmes" evidenciou como os conceitos teóricos se traduzem diretamente em código limpo, escalável e alinhado com as demandas reais das empresas de tecnologia.

7. REFERÊNCIAS BIBLIOGRÁFICAS

REACT. **React: A JavaScript library for building user interfaces**. Documentação oficial global. Disponível em: <https://react.dev/>. Acesso em: 02 set. 2026.

REACT BRASIL. **React: Uma biblioteca JavaScript para criar interfaces de usuário**. Documentação oficial em português. Disponível em: <https://pt-br.react.dev/>. Acesso em: 02 set. 2026.

REACT DOCS. **Descrevendo a UI – React**. Seção de conceitos fundamentais e JSX. Disponível em: <https://pt-br.react.dev/learn/describing-the-ui>. Acesso em: 02 set. 2026.

REACT DOCS. **Built-in React Hooks**. Referência completa da API de Hooks. Disponível em: <https://react.dev/reference/react/hooks>. Acesso em: 02 set. 2026.

REACT FOUNDATION. **The React Foundation Announcement**. Blog oficial do React. Disponível em: <https://pt-br.react.dev/blog/2026/02/24/the-react-foundation>. Acesso em: 02 set. 2026.

SILVA, Maurício Samy. **React: Construindo aplicações web de alta performance**. São Paulo: Novatec Editora, 2021.